

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12



Disusun Oleh:
Wisnu Bimantoro 105224045

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Pada praktikum Modul 12 saya mempelajari konsep SOLID Principles dalam Pemrograman Berorientasi Objek (OOP). SOLID merupakan kumpulan lima prinsip desain perangkat lunak yang bertujuan agar kode menjadi lebih mudah dipahami, mudah dikembangkan, dan tidak mudah rusak ketika terjadi perubahan kebutuhan sistem.

Pada THT ini saya menganalisis permasalahan yang terjadi pada sistem pengisian Kartu Rencana Studi (KRS) kampus yang memiliki sebuah kelas besar bernama SistemKRSManger. Kelas tersebut menangani terlalu banyak tugas sekaligus seperti validasi prasyarat, perhitungan UKT, pembuatan PDF, dan penyimpanan data ke database. Untuk mengatasi masalah tersebut dilakukan perancangan ulang sistem menggunakan prinsip Single Responsibility Principle (SRP), Open Closed Principle (OCP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP) sehingga sistem menjadi lebih fleksibel dan mudah dikembangkan.

II. Variabel

No	Variabel	Tipe Data	Fungsi
1	database	DatabaseC onnection	Menyimpan objek koneksi database
2	calculator	UKTCalcu lator	Menyimpan objek perhitungan UKT
3	sks	int	Menyimpan jumlah SKS mahasiswa
4	totalUKT	double	Menyimpan hasil perhitungan UKT
5	data	String	Menyimpan data KRS yang akan disimpan
6	nim	String	Menyimpan NIM mahasiswa
7	namaMataKuliah	String	Menyimpan nama mata kuliah
8	database	DatabaseC onnection	Dependency untuk penyimpanan data
9	calculator	UKTCalcu lator	Dependency untuk menghitung UKT

III. Constructor dan Method

No	Nama Method	Jenis	Fungsi
1	saveKRS()	Method	Menyimpan data KRS
2	validate()	Method	Memvalidasi prasyarat mahasiswa
3	generatePDF()	Method	Membuat file PDF KRS
4	calculateUKT()	Method	Menghitung biaya UKT
5	tampilInfo()	Method	Menampilkan informasi mata kuliah
6	alokasiAsistenLab()	Method	Mengalokasikan asisten laboratorium
7	cekPeralatanPraktikum()	Method	Memeriksa peralatan praktikum
8	KRSService()	Constructor	Menginisialisasi dependency
9	prosesKRS()	Method	Memproses pengisian KRS
10	main()	Static Method	Menjalankan program

IV. Pembahasan Code

1. Penjelasan Class Interface DatabaseConnection

```
public interface DatabaseConnection {  
    void saveKRS(String data);  
}
```

Penjelasan:

Pada bagian ini digunakan sebagai abstraksi untuk proses penyimpanan data KRS. Dengan menggunakan interface, sistem tidak bergantung langsung pada jenis database tertentu. Prinsip ini menerapkan **Dependency Inversion Principle (DIP)** karena class tingkat tinggi tidak bergantung pada implementasi database secara langsung.

2. Pembahasan Class MySQLDatabaseConnection

```
public class MySQLDatabaseConnection
    implements DatabaseConnection {

    @Override
    public void saveKRS(String data) {
        System.out.println("Menyimpan data KRS ke MySQL");
    }
}
```

Pembahasan:

Class ini merupakan implementasi dari interface DatabaseConnection yang digunakan ketika sistem masih menggunakan database MySQL. Method saveKRS() bertugas menyimpan data KRS ke database.

3. Pembahasan Class NoSQLDatabaseConnection

```
public class NoSQLDatabaseConnection
    implements DatabaseConnection {

    @Override
    public void saveKRS(String data) {
        System.out.println("Menyimpan data KRS ke Cloud NoSQL");
    }
}
```

Pembahasan

Class ini digunakan ketika sistem bermigrasi ke Cloud NoSQL. Karena menggunakan interface yang sama, proses migrasi dapat dilakukan tanpa mengubah class lain pada sistem.

4. Penjelasan Class PrerequisiteValidator

```
public class PrerequisiteValidator {
```

```

    public boolean validate(String nim) {
        System.out.println("Memvalidasi prasyarat mahasiswa");
        return true;
    }
}

```

Penjelasan

Class ini memiliki satu tanggung jawab yaitu melakukan validasi prasyarat mahasiswa sebelum mengambil mata kuliah. Pemisahan tugas ini merupakan penerapan Single Responsibility Principle (SRP).

5. Pembahasan PDFGenerator

```

public class PDFGenerator {

    public void generatePDF() {
        System.out.println("Membuat file PDF KRS");
    }
}

```

Penjelasan Kode

Class ini hanya bertanggung jawab membuat file PDF KRS. Jika format PDF berubah, hanya class ini yang perlu diperbarui tanpa memengaruhi proses lain dalam sistem.

6. Pembahasan Class Interface UKTCalculator

```

public interface UKTCalculator {
    double calculateUKT(int sks);
}

```

Interface ini digunakan sebagai kontrak untuk seluruh metode perhitungan UKT. Dengan adanya interface ini, sistem dapat menambahkan jenis perhitungan baru tanpa mengubah kode lama sehingga memenuhi prinsip Open Closed Principle (OCP).

7. Pembahasan Class RegulerUKTCalculator

```

public class RegulerUKTCalculator
    implements UKTCalculator {

```

```

@Override
public double calculateUKT(int sks) {
    return sks * 150000;
}
}

```

Penjelasan:

Class ini digunakan untuk menghitung UKT mahasiswa jalur reguler berdasarkan jumlah SKS yang diambil.

8. Pembahasan Class KaryawanUKTCalculator

Penjelasan serupa dengan RegulerUKTCalculator, tetapi digunakan untuk jalur karyawan dengan aturan perhitungan yang berbeda.

9. Pembahasan Class InternasionalUKTCalculator

Class ini menghitung UKT mahasiswa internasional dengan tarif yang berbeda dari jalur lainnya.

10. Pembahasan Class BidikmisiUKTCalculator

Class ini digunakan untuk mahasiswa penerima Bidikmisi sehingga hasil perhitungan UKT bernilai nol.

11. Pembahasan Class MBKMUUKTCalculator

Class ini dibuat untuk mendukung program MBKM tanpa perlu mengubah class calculator yang sudah ada sebelumnya.

12. Pembahasan Interface MataKuliah

```

public interface MataKuliah {
    void tampilInfo();
}

```

Penjelasan

Interface ini berisi operasi dasar yang dimiliki seluruh jenis mata kuliah.

13. Pembahasan Interface Praktikum

```

public interface Praktikum {
    void alokasiAsistenLab();
    void cekPeralatanPraktikum();
}

```

Penjelasan

Interface ini hanya digunakan oleh mata kuliah yang memiliki kegiatan praktikum sehingga memenuhi Interface Segregation Principle (ISP).

14. Pembahasan Class MataKuliahTeori

Class ini hanya mengimplementasikan interface MataKuliah karena tidak memerlukan fitur laboratorium.

15. Pembahasan Class MataKuliahPraktikum

Class ini mengimplementasikan interface MataKuliah dan Praktikum karena memiliki kebutuhan praktikum seperti alokasi asisten laboratorium dan pemeriksaan peralatan.

16. Pembahasan Class MataKuliahKKN

Class ini hanya mengimplementasikan MataKuliah karena tidak memiliki aktivitas praktikum.

17. Pembahasan Class KRSService

```
public class KRSService {  
  
    private DatabaseConnection database;  
    private UKTCalculator calculator;  
  
    public KRSService(DatabaseConnection database,  
                      UKTCalculator calculator) {  
  
        this.database = database;  
        this.calculator = calculator;  
    }  
  
    public void prosesKRS(int sks) {  
  
        double totalUKT =  
            calculator.calculateUKT(sks);  
  
        System.out.println("Total UKT : Rp"  
                           + totalUKT);  
  
        database.saveKRS("Data KRS Mahasiswa");  
    }  
}
```

Penjelasan

Class ini merupakan service utama yang mengelola proses KRS. Kelas ini tidak membuat objek database secara langsung, tetapi menerima dependency melalui constructor. Teknik ini disebut Dependency Injection dan merupakan implementasi Dependency Inversion Principle (DIP).

18. Pembahasan Class Main

```
DatabaseConnection database =
```

```
new NoSQLDatabaseConnection();

UKTCalculator calculator =
    new MBKMUKTCalculator();

KRSService service =
    new KRSService(database, calculator);

service.prosesKRS(20);
```

Penjelasan

Pada bagian ini sistem menggunakan database NoSQL dan perhitungan UKT MBKM. Jika ingin menggunakan database atau skema UKT lain, programmer cukup mengganti objek yang digunakan tanpa mengubah class KRSService.

V. Kesimpulan

Kesimpulan dari praktikum Modul 12 ini adalah saya berhasil memahami penerapan SOLID Principles pada Java. Melalui proses analisis dan redesign sistem KRS kampus, saya dapat memahami bagaimana memisahkan tanggung jawab class menggunakan Single Responsibility Principle, menambahkan fitur baru menggunakan Open Closed Principle, memisahkan interface menggunakan Interface Segregation Principle, serta mengurangi ketergantungan antar class menggunakan Dependency Inversion Principle. Dengan menerapkan prinsip-prinsip tersebut, sistem menjadi lebih fleksibel, mudah dipelihara, dan siap dikembangkan untuk kebutuhan di masa mendatang.

VI. Daftar Pustaka

Deitel, P. J., & Deitel, H. M. (2017). *Java How to Program, Early Objects* (11th ed.). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/java-how-to-program-early-objects/P200000003310/9780134743356>

Farrell, J. (2022). *Java Programming* (10th ed.). Cengage Learning. <https://www.cengage.com/c/java-programming-10e-farrell/9780357675137/>

Oracle. (n.d.). *Control Flow Statements*. The Java™ Tutorials. <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>

Tim Asisten Praktikum. (2026). *Modul 3: Java Basics II*. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.

W3Schools. (n.d.). *Java If...Else*. https://www.w3schools.com/java/java_conditions.asp